



Nomes: Elisa Belquis e Miguel Estivalet
Disciplina: INE5408 - Estruturas de Dados
Professores: Alexandre Gonçalves Silva e
Aldo Von Wangenheim
Data: 02/12/2025

PROJETO II

1. INTRODUÇÃO

O segundo projeto consistia em aplicar uma estrutura hierárquica diferente das abordadas ao longo dos meses finais da disciplina Estrutura de Dados. A Trie, também chamada de árvore digital, é uma estrutura de dados especializada em organizar conjuntos de strings de maneira hierárquica. Diferente de métodos que armazenam a palavra completa isoladamente, a Trie aproveita os caracteres em comum entre as entradas para economizar memória e acelerar significativamente operações essenciais como busca, inserção e exclusão.

O funcionamento dessa estrutura baseia-se em um sistema de nós, onde tudo se inicia em uma raiz vazia e cada conexão subsequente representa um caractere específico. Dessa forma, cada caminho percorrido na árvore corresponde a uma palavra armazenada. A grande eficiência da Trie surge justamente dessa arquitetura, pois palavras que começam com as mesmas letras (prefixos) compartilham os mesmos nós iniciais, criando uma forma compacta de armazenamento que evita redundâncias desnecessárias.

Este relatório detalha a implementação de um programa projetado para a indexação e recuperação eficiente de palavras em grandes arquivos de dicionários, os quais foram disponibilizados pelos professores no formato *.dic*. O problema central é formado por duas tarefas principais que foram implementadas por meio da linguagem C++.

Primeiramente, a construção da trie a partir das palavras existentes nos arquivos e, então, o retorno da existência ou não deste prefixo digitado dentro da árvore e sua quantidade, quando houver correspondência. Segundamente, deve-se indicar a localização da palavra no arquivo e o tamanho da linha que a define. Tendo ambos os problemas definidos, as linhas que deverão ser produzidas na saída padrão são: a mensagem 'P is prefix of N words' na primeira linha, e 'P is in (D,C)' na linha seguinte (sendo D a posição, e C o comprimento).

2. DESCRIÇÃO DOS ALGORITMOS PRINCIPAIS

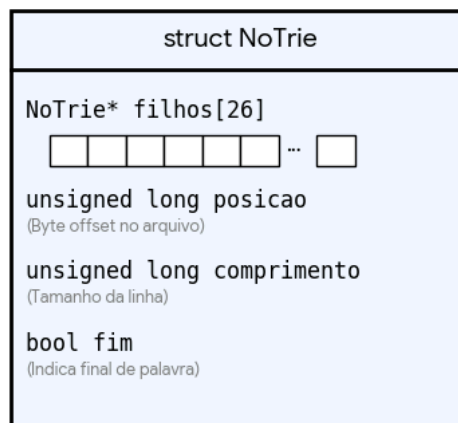
A solução foi arquitetada em torno da estrutura *NoTrie* e dividida em três processos lógicos principais: a estruturação dos nós, o carregamento/mapeamento do dicionário e a busca com contagem recursiva.

2.1. ESTRUTURA DO NÓ

A base da árvore é o *struct NoTrie* (Figura 1), projetado para conter tanto os ponteiros de navegação quanto os metadados do arquivo. Ele possui o mapeamento de filhos que consiste no uso de um vetor estático *NoTrie* filhos[26]* para representar as letras de 'a' a 'z'. O índice é calculado aritmeticamente (*char - 'a'*) para converter os códigos em ASCII para números indiciáveis dentro do array.

Para resolver o problema de indexação, cada nó possui os campos *unsigned long posicao* e *unsigned long comprimento*. Estes campos só são preenchidos se o nó marcar o fim de uma palavra válida e ajudam a trazer os metadados de arquivo. Por fim, um booleano *fim* indica se o caminho da raiz até o nó atual forma uma palavra completa registrada no dicionário, gerando uma flag.

Figura 1: Ilustração da estrutura de nó implementada



Fonte: os autores

2.2. CARREGAMENTO E INDEXAÇÃO DO DICIONÁRIO

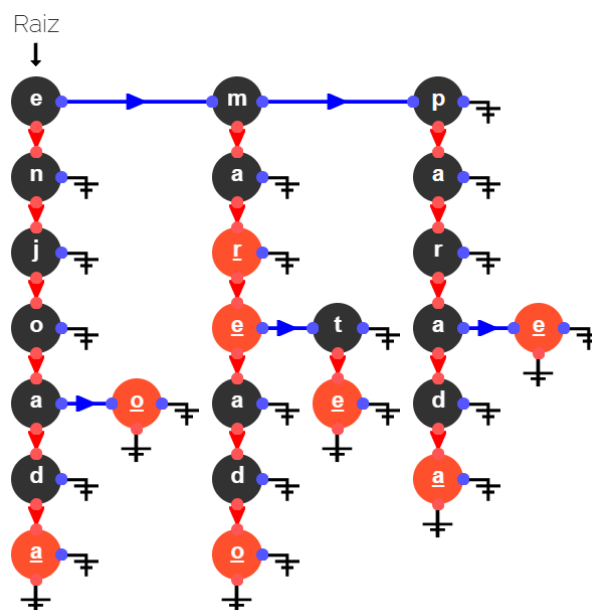
A função *carregarDicionario* é responsável por construir a Trie e, simultaneamente, mapear o arquivo físico. A lógica segue os passos:

1. Leitura Sequencial: O arquivo é aberto e lido linha a linha. O programa mantém o controle da posição do cursor de leitura (*file pointer*) usando *tellg()*.

2. Extração: Para cada linha, o algoritmo identifica a palavra-chave observando os delimitadores '[' e ']’.
3. Inserção na Trie: A função *inserir* é chamada passando a palavra, a posição inicial da linha e seu tamanho total.
 - a. O algoritmo percorre cada caractere da palavra.
 - b. Calcula-se o índice (0-25). Se o ponteiro correspondente for nulo, um novo *NoTrie* é alocado.
 - c. Ao chegar no último caractere, a flag *fin* é setada para *true* e os metadados de posição e comprimento são gravados no nó.
4. Atualização de Cursor: Após a inserção, a posição do ponteiro de arquivo é atualizada para o início da próxima linha, garantindo que o índice aponte corretamente para o próximo verbete.

Na Figura 2, foi demonstrado a estrutura de uma Trie após a inclusão das seguintes palavras: “enjoo”, “enjoada”, “pare”, “parada”, “mar”, “maré”, “mareado”, “marte”. Ela foi gerada por meio de um simulador da Unesp denominado Cadilag.

Figura 2: Árvore Trie experimental



Fonte: os autores via Cadilag

2.2. BUSCA DE PREFIXO E CONTAGEM RECURSIVA (DFS)

Para responder à consulta "P is prefix of N words", não armazenamos contadores estáticos. Em vez disso, utilizamos uma abordagem de contagem dinâmica via Busca em Profundidade (DFS) na função *contaPalavras*. A função

buscaPrefixo percorre a árvore seguindo os caracteres da palavra de entrada. Sua lógica de execução é:

- Se em algum momento o ponteiro para o próximo caractere for *nullptr*, o prefixo não existe. O programa retorna "is not prefix".
- Se a navegação for bem sucedida, o ponteiro para o nó final do prefixo é retornado.

A Contagem Recursiva (Algoritmo DFS) depende do nó retornado. A função *contaPalavras* é invocada para explorar toda a subárvore. E possui os seguintes passos:

1. Passo Base: Se o nó for nulo, retorna 0;
2. Passo de Acumulação: Uma variável total é iniciada. Se o nó atual tiver fim == true, total começa com 1 (pois o próprio prefixo é uma palavra). Caso contrário, começa com 0;
3. Passo Recursivo: O algoritmo itera sobre todos os 26 filhos possíveis. Para cada filho existente, chama-se *contaPalavras* novamente, somando o retorno ao total;
4. Retorno: A soma final representa quantas palavras "nascem" daquele nó.

3. CONCLUSÃO E DIFICULDADES ENCONTRADAS

O desenvolvimento do projeto permitiu a aplicação prática de estruturas de dados hierárquicas para problemas de indexação e recuperação de texto. A escolha por calcular a quantidade de palavras prefixadas dinamicamente (via recursão), em vez de armazenar um contador em cada nó durante a inserção, mostrou-se uma decisão de projeto interessante: reduziu o consumo de memória RAM da estrutura, embora tenha aumentado ligeiramente a complexidade de tempo da operação de busca (*contaPalavras*), já que é necessário percorrer a subárvore a cada consulta. A Trie provou ser extremamente eficiente para validar a existência de palavras e recuperar seus metadados, operando com complexidade proporcional ao tamanho da palavra buscada.

Entre as dificuldades enfrentadas, destaca-se a manipulação correta dos fluxos de arquivo em C++. Sincronizar a leitura da linha (*getline*) com a obtenção da posição correta do cursor (*tellg*) para a próxima iteração exigiu atenção, pois a posição deve corresponder ao início exato da linha da palavra, e não ao meio ou fim.

Além disso, a implementação da recursão para a contagem de palavras exigiu cuidado com o caso base (nós nulos) para evitar falhas de segmentação e garantir que a contagem incluísse corretamente o próprio nó do prefixo, caso ele também fosse uma palavra válida. A gestão de memória dinâmica com ponteiros também foi um ponto crítico para garantir a integridade da árvore.

4. REFERÊNCIAS

EOFILOFF, Paulo. **Árvores digitais (tries)**. São Paulo: IME-USP, [s.d.]. Disponível em: <https://www.ime.usp.br/~pf/estruturas-de-dados/aulas/tries.html>. Acesso em: 29 nov. 2025.

GEEKSFORGEEEKS. **Trie data structure**: insert and search. [S.l.: s.n.], [s.d.]. Disponível em: <https://www.geeksforgeeks.org/trie-insert-and-search/>. Acesso em: 29 nov. 2025

CVR BIOINFORMATICS. **Trie data structure**. [S.l.], 2 dez. 2014. Disponível em: <https://bioinformatics.cvr.ac.uk/trie-data-structure/>. Acesso em: 29 nov. 2025.

UNIVERSIDADE ESTADUAL PAULISTA. Faculdade de Ciências e Tecnologia. **Cadilag**: acesso livre. Presidente Prudente, [s.d.]. Disponível em: https://portaldoprofessor.fct.unesp.br/projetos/cadilag/acesso_livre.php. Acesso em: 30 nov. 2025.